



Libc Functions

So far, we have only been printing Fibonacci numbers that are less than `10`. But this way, our program is static and will print the same output every time. To make it more dynamic, we can ask the user for the max Fibonacci number they want to print and then use it with `cmp`. Before we start, let's recall the function calling convention:

1. `Save Registers` on the Stack (`Caller Saved`)
2. Pass `Function Arguments` (like syscalls)
3. Fix `Stack Alignment`
4. Get Functions' `Return Value` (in `rax`)

So, let's import our function and start with the calling convention steps.

Importing libc Functions

To do so, we can use the `scanf` function from `libc` to take user input and have it properly converted to an integer, which we will later use with `cmp`. First, we must import the `scanf`, as follows:

```
global _start
extern printf, scanf
```

We can now start writing a new procedure, `getInput`, so we can call it when we need to:

```
getInput:
    ; call scanf
```

nasm

Saving Registers

As we are at the beginning of our program and have not yet used any register, we don't have to worry about saving registers to the Stack. So, we can proceed with the second point, and pass the required arguments to `scanf`.

Function Arguments

Next, we need to know what arguments are accepted by `scanf`, as follows:

```
kimvutrinh@htb[/htb]$ man -s 3 scanf
```

```
...SNIP...
```

```
int scanf(const char *format, ...);
```

shellsession

We see that similarly to `printf`, `scanf` accepts an input format and the buffer we want to save the user input into. So, let's first add the `inFormat` variable:

```
nasm
section .data
    message db "Please input max Fn", 0x0a
    outFormat db "%d", 0x0a, 0x00
    inFormat db "%d", 0x00
```

We also changed our intro message from `Fibonacci Sequence:` to `Please input max Fn`, to tell the user what input is expected from them.

Next, we must set a buffer space for the input storage. As we mentioned in the `Processor Architecture` section, uninitialized buffer space must be stored in the `.bss` memory segment. So, at the beginning of our assembly code, we must add it under the `.bss` label, and use `resb 1` to tell `nasm` to reserve 1 byte of buffer space, as follows:

```
nasm
section .bss
    userInput resb 1
```

We can now set our function arguments under our `getInput` procedure:

```
nasm
getInput:
    mov rdi, inFormat ; set 1st parameter (inFormat)
    mov rsi, userInput ; set 2nd parameter (userInput)
```

Stack Alignment

Next, we have to ensure that a 16-bytes boundary aligns our Stack. We are currently inside the `getInput` procedure, so we have 1 `call` instruction and no `push` instructions, so we have an 8-byte boundary. So, we can use `sub` to fix `rsp`, as follows:

```
getInput:
    sub rsp, 8
    ; call scanf
    add rsp, 8
```

nasm

We can `push rax` instead, and this will properly align the Stack as well. This way, our Stack should be perfectly aligned with a 16-byte boundary.

Function Call

Now, we set the function arguments and `call scanf`, as follows:

```
getInput:
    sub rsp, 8           ; align stack to 16-bytes
    mov rdi, inFormat    ; set 1st parameter (inFormat)
    mov rsi, userInput   ; set 2nd parameter (userInput)
    call scanf           ; scanf(inFormat, userInput)
    add rsp, 8           ; restore stack alignment
    ret
```

nasm

We will also add `call getInput` at `_start`, so that we go to this procedure right after printing the intro message, as follows:

```
nasm
section .text
_start:
    call printMessage    ; print intro message
    call getInput        ; get max number
    call initFib         ; set initial Fib values
    call loopFib         ; calculate Fib numbers
    call Exit            ; Exit the program
```

Finally, we have to make use of the user input. To do so, instead of using a static `10` when comparing in `cmp rbx, 10`, we will change it to `cmp rbx, [userInput]`, as follows:

```
nasm
loopFib:
    ...SNIP...
    cmp rbx,[userInput] ; do rbx - userInput
    js loopFib          ; jump if result is <0
    ret
```

Note: We used `[userInput]` instead of `userInput`, as we wanted to compare with the final value, and not with the pointer address.

With all of that done, our final complete code should look as follows:

```
nasm
global _start
extern printf, scanf
```

```

section .data
    message db "Please input max Fn", 0x0a
    outFormat db "%d", 0x0a, 0x00
    inFormat db "%d", 0x00

section .bss
    userInput resb 1

section .text
_start:
    call printMessage    ; print intro message
    call getInput        ; get max number
    call initFib         ; set initial Fib values
    call loopFib         ; calculate Fib numbers
    call Exit            ; Exit the program

printMessage:
    ...SNIP...

getInput:
    sub rsp, 8           ; align stack to 16-bytes
    mov rdi, inFormat    ; set 1st parameter (inFormat)
    mov rsi, userInput   ; set 2nd parameter (userInput)
    call scanf           ; scanf(inFormat, userInput)
    add rsp, 8           ; restore stack alignment
    ret

initFib:
    ...SNIP...

printFib:
    ...SNIP...

```

```
loopFib:
    ...SNIP...
    cmp rbx,[userInput] ; do rbx - userInput
    js loopFib          ; jump if result is <0
    ret

Exit:
    ...SNIP...
```

Dynamic Linker

Let's assemble our code, link it, and try to print Fibonacci numbers up to 100 :

```
shellsession
kimvutrinh@htb[/htb]$ nasm -f elf64 fib.s && ld fib.o -o fib -lc --dynamic-linker
/lib64/ld-linux-x86-64.so.2 && ./fib
```

Please input max Fn:

100

1

1

2

3

5

8

13

21

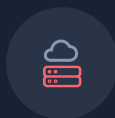
34

55
89

We see that our code worked as expected and printed Fibonacci numbers less than the number we specified. With this, we can complete our module project and create a program that calculates and prints Fibonacci numbers based on an input we provide, using nothing but assembly.

Besides, we need to learn how to turn Assembly code into machine shellcode, which we can then use directly in our payloads in Binary Exploitation.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ

PRO

Optional Exercise 1

XP +40 ^

The current string format we are using only allows numbers up to 2 billion. What format can we use to allow up to 3 billion? "Check length modifiers in the 'printf' man page"

Write your answer

Submit



Reveal Answer

← Previous

Section 20 / 24

 +20

Next

Mark Complete & Next